Daniel Copete
Tema 1
Despliegue

1.Modelo Arquitectónico General de Spotify :

Qué modelo usa: Spotify se basa en una arquitectura de **microservicios**. En lugar de una aplicación monolítica, la plataforma se divide en cientos de servicios pequeños e independientes (búsqueda, recomendaciones, playlists, reproducción, perfiles...). Se comunican entre sí mediante llamadas directas (gRPC/HTTP) y mensajería de **eventos**, por ejemplo para registrar lo que escucha cada usuario y alimentar las recomendaciones. Todo corre en contenedores sobre la nube de Google Cloud, y cada servicio pertenece a un equipo autónomo (*squad*) que lo desarrolla y mantiene.

Por qué es adecuado:

- **Escalabilidad:** cada servicio escala por separado. El de streaming necesita muchas más instancias que el de ajustes de cuenta, así que solo se amplía lo necesario.
- **Resiliencia:** si un servicio falla, el resto sigue funcionando. Si caen las recomendaciones, el usuario aún puede reproducir música.
- **Desarrollo ágil:** equipos pequeños despliegan sus servicios de forma independiente y frecuente, sin depender de un despliegue único y gigante.

Inconveniente a mencionar: mayor complejidad operativa y dificultad para depurar sistemas distribuidos. Spotify lo gestiona con herramientas internas como Backstage.

Fuentes:

https://www.okoone.com/spark/technology-innovation/how-spotify-built-the-tech-behind-streaming-millions-of-songs/

2.Tecnologías Front-end:

Tecnologías front-end

Web y escritorio

- El reproductor web es una aplicación de una sola página (SPA) que se apoya en la Spotify Web API, que agrupa el acceso a muchos microservicios en una interfaz unificada. El nuevo reproductor web se basa en React y Redux.
- La app de escritorio es una aplicación nativa para Windows y Mac que usa CEF (Chromium Embedded Framework) para mostrar una interfaz basada en web. Es decir, por dentro es un navegador embebido.
- Spotify unificó ambos clientes: la arquitectura final tiene una capa de APIs de plataforma y una interfaz en React que las consume mediante React Hooks, con acceso al back-end vía GraphQL y Web API. Antes, cada “página” del escritorio era una mini-aplicación independiente dentro de su propio iframe.

Móvil (iOS y Android)

Las apps móviles son nativas. Las ofertas de empleo oficiales de Spotify piden experiencia con Kotlin para Android y con Swift u Objective-C para iOS, y mencionan C++ como un plus para

integrar código nativo. Es una fuente indirecta, así que en el trabajo preséntalo como “según sus ofertas de empleo”.

Cómo contribuyen a la experiencia de usuario

- **React y los componentes reutilizables** permiten una interfaz coherente entre web y escritorio y actualizaciones rápidas de la pantalla al cambiar la canción, la cola o las playlists, lo que da sensación de interactividad inmediata.
- **Compartir la interfaz entre web y escritorio** acelera el desarrollo y evita diferencias de comportamiento entre plataformas. Spotify lo cuenta como motivo de la unificación: los ingenieros nuevos se adaptaban mejor al código del reproductor web que al del escritorio.
- **Las apps nativas** (Kotlin, Swift) dan mejor rendimiento y acceso al sistema operativo: reproducción en segundo plano, notificaciones, control desde la pantalla de bloqueo, etc.
- **GraphQL y la Web API** simplifican que el cliente pida solo los datos que necesita, lo que reduce tiempos de carga.

Fuentes:

https://engineering.atspotify.com/2021/4/building-the-future-of-our-desktop-apps

3.Tecnologías Back-end:

Lenguajes del lado del servidor

- **Java** es el lenguaje principal de los servicios (la lógica de negocio). Lo respalda el artículo de Devlane, que lo describe como el lenguaje principal de Spotify, y también las ofertas de empleo y artículos de ingeniería que vimos antes. Es una fuente secundaria, así que apóyalo además en un artículo oficial si lo encuentras.
- **Python** tiene un papel histórico en datos. Spotify empezó como una empresa de Python y creó Luigi para orquestar trabajos y ejecutar jobs de MapReduce en Python sobre Hadoop.

- **Scala** es el lenguaje preferido para el procesamiento de datos. Spotify creó Scio, una API en Scala para Apache Beam y Google Cloud Dataflow, y lo convirtió en su framework de procesamiento de datos preferido, con pipelines por lotes y en streaming. Justifican Scala porque, según ellos, el código de los pipelines en Scala suele ocupar un 20 % de su equivalente en Java con rendimiento comparable.

Frameworks y tecnologías de servidor

- **Apollo:** un framework Java propio para servicios, parecido a Spring Boot. Lo he visto solo en un artículo de divulgación, no en documentación oficial, así que verifícalo o cítalo con cautela.
- **Kafka y Hadoop** aparecen en las fuentes de divulgación como base del procesamiento de eventos y datos.
- **Orquestación de pipelines:** la mayoría de sus pipelines usaban Luigi (para Python) y Flo (para Java), y en 2019 el equipo decidió sustituirlos por Flyte, por su extensibilidad, soporte de varios lenguajes y escalabilidad.

Fuentes:
https://engineering.atspotify.com/2017/10/big-data-processing-at-spotify-the-road-to-scio-part-1

4. Sistemas de Bases de Datos:
Spotify no usa una sola base de datos, sino varias según el tipo de dato. Las fuentes que he encontrado permiten sostener esto:

Relacional (PostgreSQL)

- Spotify usó Postgres como almacenamiento principal de usuarios durante años. Es una base de datos muy ocupada y una pieza central de la infraestructura, y sirvió de maestro mientras se probaba Cassandra en paralelo. Es el modelo clásico para datos estructurados y transaccionales, pero le costaba escalar a su volumen de usuarios.

NoSQL, columnas anchas (Cassandra)

- En 2015 todos los usuarios de Spotify pasaron a almacenarse en Cassandra en lugar de Postgres. Se eligió por su escalabilidad horizontal y tolerancia a fallos, útiles cuando hay millones de usuarios repartidos por el mundo. Esa misma fuente cuenta una dificultad interesante: garantizar que cada nombre de usuario pertenezca a un solo usuario obligó a usar transacciones ligeras (LWT), costosas en Cassandra. Es un buen ejemplo de trade-off que puedes citar.
- Según una fuente secundaria, Cassandra almacena datos de usuario como playlists y bibliotecas musicales.

Bases de datos gestionadas de Google Cloud (Bigtable y Datastore)

- Al migrar a la nube, Spotify indicó que implementaba Google Cloud Datastore y Google Cloud Bigtable para el almacenamiento. Son NoSQL gestionadas: Spotify delega su operación y se centra en los servicios.

Series temporales (Heroic)

- Heroic es un sistema de series temporales creado en Spotify, basado en Cassandra o Bigtable y en Elasticsearch. Sirve para almacenar y consultar métricas de monitorización a lo largo del tiempo. Esta información viene de DB-Engines, un ranking de bases de datos, así que es una fuente de apoyo y no oficial.

Caché en memoria (Redis)

- Una fuente secundaria afirma que Spotify usa Redis para cachear metadatos de canciones, álbumes y artistas. Presenta esto con cautela: no lo he podido confirmar en una fuente oficial.

https://engineering.atspotify.com/2015/6/user-database-switch

5. Comunicación entre Servicios y APIs:

Comunicación entre servicios y APIs

Evolución del protocolo interno: de Hermes a gRPC

- **Hermes (protocolo propio, histórico).** Es una pila de mensajería creada en Spotify sobre ZeroMQ, protobuf y gevent. Permitía comunicación petición/respuesta y también publicación/suscripción (por ejemplo, un cliente suscrito a los cambios de una playlist). Se creó antes de HTTP/2 y gRPC, lo que explica por qué desarrollaron su propio protocolo.
- **gRPC (el estándar actual).** Spotify construyó históricamente sus servicios sobre su propio protocolo y framework de mensajería, y emprendió la migración de más de 3.000 servicios a gRPC. gRPC ofrece servicios y mensajes con tipado fuerte (definidos con protobuf), lo que facilita la coordinación entre cientos de equipos autónomos. Una de las fuentes que usé lo resume bien: el reto de la migración fue sobre todo organizativo, no solo técnico.

APIs hacia los clientes (web y móvil)

- La Spotify Web API combina el acceso a muchos microservicios en una interfaz unificada para manipular los datos de Spotify. Así, los clientes no hablan con cientos de servicios, sino con una capa de entrada.
- Además, la interfaz de web y escritorio accede al back-end mediante servicios GraphQL y Web API. GraphQL permite al cliente pedir solo los datos que necesita.

Mensajería asíncrona y eventos

- Para el flujo de eventos y datos, Spotify usa mensajería tipo publicación/suscripción. En la migración a la nube, usaba Cloud Pub/Sub de Google para recoger y reenviar todos los eventos a su ecosistema. Esto sostiene la arquitectura orientada a eventos que mencionamos en el punto 1.
- Kafka aparece citado en fuentes de divulgación como pieza de la arquitectura de eventos, pero no he encontrado una fuente oficial reciente que lo confirme, así que si lo mencionas, indica que es información de fuentes secundarias.

Fuentes:
https://www.designgurus.io/answers/detail/discuss-spotify-system-architecture

6. Infraestructura y Despliegue:
Plataforma de nube: Google Cloud

- Spotify no nació en la nube: a principios de 2015 empezó a explorar una estrategia cloud y acabó comprometiéndose con Google Cloud Platform (GCP). Antes usaba centros de datos propios. a
- Completó la migración desde sus centros de datos a Google Cloud en 2016-2017. Una fuente de prensa especializada precisa que se movieron cerca de 1.200 microservicios, y que los equipos pararon el desarrollo de producto durante un sprint de dos semanas para migrar sus servicios.
- El motivo, según la dirección de ingeniería de la época, era que seguir el ritmo de la demanda de escalado exigía cada vez más esfuerzo en infraestructura propia. Spotify prefería que sus equipos se centraran en el producto.

Contenedores y orquestadores:

- Spotify fue pionera en microservicios y Docker: tenía microservicios en contenedores sobre su flota de máquinas virtuales desde 2014, con un orquestador propio de código abierto llamado Helios.

- Después pasó a Kubernetes. Según el caso de estudio de la propia documentación de Kubernetes, el equipo dedicó buena parte de 2018 a resolver los problemas técnicos de la migración, que empezó a finales de ese año y se aceleró en 2019. Otra fuente indica que la decisión de adoptar Google Kubernetes Engine se tomó a finales de 2017.

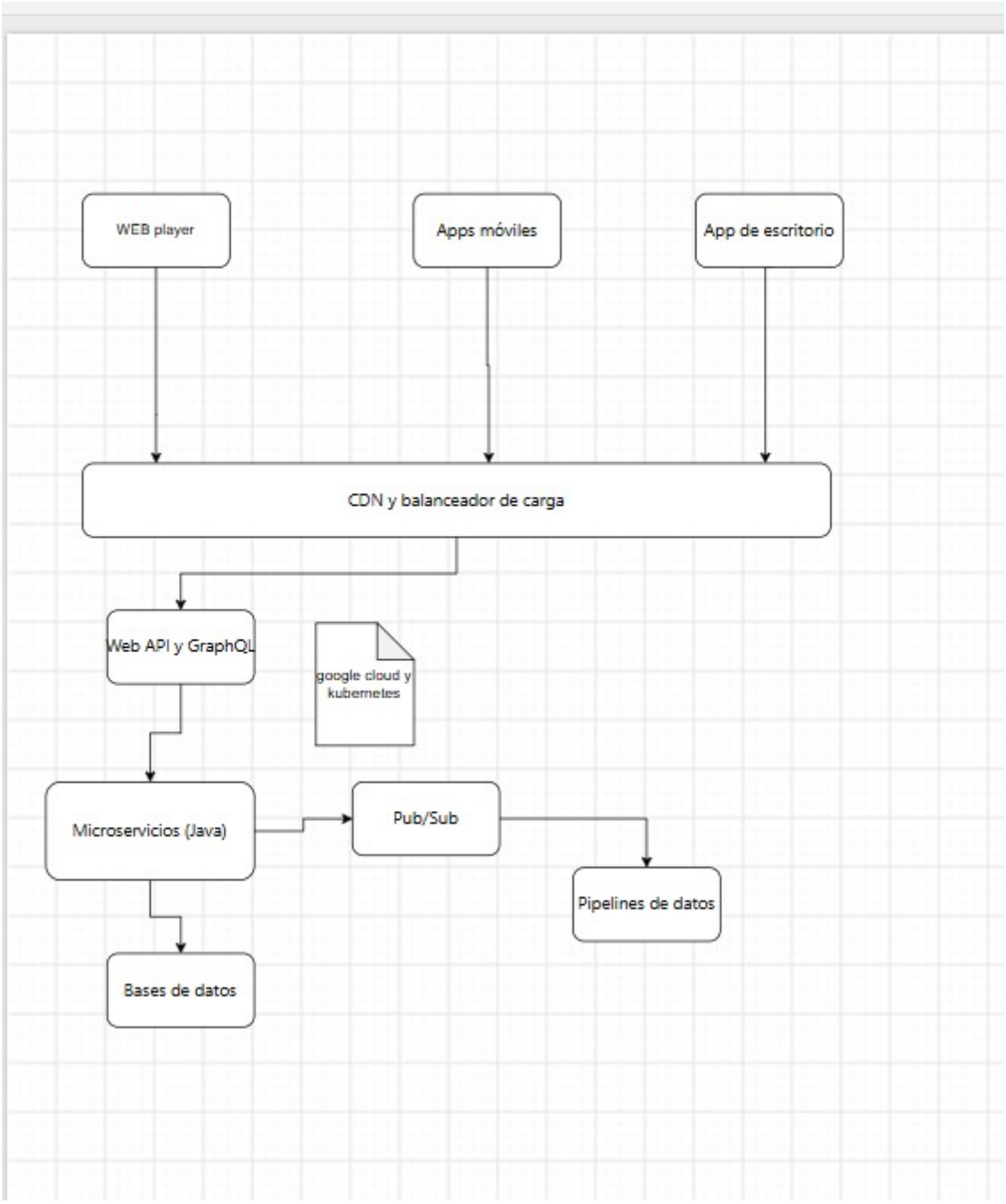
- Una fuente secundaria (Altoros) habla de más de 1.600 servicios en producción sobre Kubernetes, con unos 100 microservicios nuevos al mes, monitorizados con Backstage. Es un dato de una fecha concreta, preséntalo como tal.

Cómo contribuyen a la escalabilidad y disponibilidad

- **Escalabilidad:** los contenedores empaquetan cada servicio con sus dependencias, y Kubernetes puede crear o eliminar réplicas según la demanda. Además, la nube permite añadir capacidad sin comprar hardware, algo clave para picos como el lanzamiento de Spotify Wrapped.
- **Disponibilidad:** Kubernetes reinicia automáticamente los contenedores que fallan y reparte el tráfico entre réplicas, de modo que la caída de una instancia no interrumpe el servicio.
- **Agilidad de despliegue:** cada equipo despliega sus servicios de forma independiente, lo que reduce el riesgo de cada cambio.
- **Menos carga operativa:** al usar servicios gestionados de Google, Spotify delega parte de la operación y se centra en el producto.

https://engineering.atspotify.com/2019/12/views-from-the-cloud-a-history-of-spotifys-journey-to-the-cloud-part-1-2

7.Diagrama


WEB player
Apps móviles
App de escritorio
CDN y balanceador de carga
Web API y GraphQL
google cloud y kubernetes
Microservicios (Java)
Pub/Sub
Pipelines de datos
Bases de datos

Explicación de cada parte (para que la reescribas con tus palabras)

- **Clientes .** Son las tres puertas de entrada. El reproductor web usa React, la app de escritorio es una aplicación nativa que embebe un navegador (CEF) con la misma interfaz en React, y las apps móviles son nativas .
- **CDN y balanceador .** . Su función típica sería repartir el tráfico entre instancias y servir contenido pesado, como el audio, desde servidores cercanos al usuario.
- **Web API y GraphQL .** Es la capa de entrada a los servicios. Los clientes hablan con ella por HTTPS en lugar de con cientos de microservicios, y GraphQL les permite pedir solo los datos que necesitan.
- **Microservicios en Java .** Contienen la lógica de negocio: búsqueda, playlists, recomendaciones, reproducción, etc. Se comunican entre sí con gRPC, que sustituyó al protocolo propio Hermes, y cada equipo despliega los suyos de forma independiente.
- **Google Cloud y Kubernetes .** Es la infraestructura sobre la que corre todo: los servicios van en contenedores y Kubernetes los escala, los reinicia si fallan y reparte la carga.
- **Bases de datos .** Cada una tiene un propósito. PostgreSQL es la relacional clásica, usada históricamente para datos de usuario. Cassandra es NoSQL y escala horizontalmente para millones de usuarios. Bigtable es la opción NoSQL gestionada de Google Cloud, que delega la operación. Heroic almacena series temporales, es decir, métricas de monitorización a lo largo del tiempo.
- **Pub/Sub .** Es el bus de mensajería asíncrona. Los servicios publican eventos (por ejemplo, lo que escucha cada usuario) sin esperar respuesta, lo que desacopla unos servicios de otros.
- **Pipelines de datos .** Consumen esos eventos para procesar grandes volúmenes de datos por lotes y en streaming, y alimentan análisis y recomendaciones.